

Iterative Improvement Algorithms for the Linear Ordering Problem

INFO-H-413 — Heuristic Optimization — Implementation Exercise 1

Université Libre de Bruxelles — 2026

Luis Brunard (577721)

1. Introduction

The Linear Ordering Problem (LOP) is a ranking problem. Given an $n \times n$ weight matrix C , the goal is to find a permutation π of n elements that maximises the sum of weights above the diagonal:

$$f(\pi) = \sum_{i < j} c_{\pi(i), \pi(j)}$$

In plain terms, we want i to appear before j whenever $c_{ij} > c_{ji}$. The problem is NP-hard, so exact methods are not practical beyond small instances. We use heuristics instead.

This exercise is about local search: start from some solution, make small improving changes, and stop when none is possible. That stopping point is a **local optimum**. The question we are trying to answer is which combination of choices — neighbourhood, pivoting rule, initialisation — gets us to the best local optimum, and how quickly.

2. Problem Description

Three design parameters control how local search behaves on the LOP.

2.1. Neighbourhood Structures

A neighbourhood defines which solutions are reachable in one step. We test three.

Transpose. Swap two adjacent elements. From a permutation of length n , there are $n - 1$ such pairs. The gain Δ of swapping positions i and $i + 1$ is:

$$\Delta_T(i) = c_{\pi(i+1), \pi(i)} - c_{\pi(i), \pi(i+1)}$$

Each evaluation is $O(1)$, making transpose by far the cheapest option.

Exchange. Swap any two elements, not necessarily adjacent. There are $\binom{n}{2}$ possible pairs, but computing each gain requires scanning everything between the two positions — $O(n)$ per move.

Insert. Remove an element from position i and reinsert it at position j , shifting what is in between. This gives $n(n - 1)$ possible moves, also at $O(n)$ each.

2.2. Pivoting Rules

Once a neighbourhood is fixed, there are two ways to pick the next move.

- **First-improvement (FI):** apply the first improving move found and restart the scan immediately.
- **Best-improvement (BI):** scan the full neighbourhood, then apply the single best improving move.

2.3. Initialisation

- **Random:** a uniformly shuffled permutation.
- **Chenery–Watanabe (CW):** rank each element by its row-sum score $R_i = \sum_{j \neq i} c_{ij}$ and sort in decreasing order. The idea is to put elements that gain the most from being ranked early at the front, giving a starting point that is already better than a random shuffle.

3. Algorithms

3.1. Iterative Improvement (Exercise 1.1)

The algorithm is simple:

- Generate an initial solution π_0 . ($\rightarrow$ Random/CW).
- While an improving neighbour exists:
 - (First-improvement) Apply the first move with $\Delta > 0$; restart scan.
 - (Best-improvement) Apply the move with the largest $\Delta > 0$.
- Return the local optimum π^* .

Combining 2 pivoting rules, 3 neighbourhoods, and 2 initialisations gives **12 configurations** in total.

3.2. Variable Neighbourhood Descent (Exercise 1.2)

The insight behind VND is that being stuck in a local optimum for one neighbourhood does not mean you are stuck for all of them. Instead of stopping, we switch to a different neighbourhood and keep searching. If an improvement is found, we go back to the first neighbourhood. We stop only when none of the neighbourhoods can improve the current solution.

We use first-improvement throughout, starting from the CW solution. Two orderings are tested:

- **VND-TEI:** Transpose $\rightarrow$ Exchange $\rightarrow$ Insert
- **VND-TIE:** Transpose $\rightarrow$ Insert $\rightarrow$ Exchange

4. Statistical Testing Method

4.1. Standard deviation

The **standard deviation** (SD) measures how spread a set of values is around their mean. A small SD means results are consistent across instances; a large one means performance depends heavily on the specific problem [1].

4.2. Wilcoxon test and p-value

Average RPDs (defined in section 5) alone can be misleading — a gap might come from a few unusual instances rather than a consistent trend. To check this, we use the **Wilcoxon signed-rank test** [2], a non-parametric test for paired data. For each instance we compute the RPD difference between two algorithms, rank those differences by absolute value, and check whether one algorithm wins consistently. The output is a **p-value** — the probability of seeing a gap this large if the two algorithms were actually equivalent. We use $\alpha = 0.05$: if $p < 0.05$, the

difference is **statistically significant** [3]. All tests were run in R with the built-in `wilcox.test` function [4].

5. Results and Analysis

Each of the 14 algorithm configurations was run once on all 78 instances. To compare solutions, we use the **Relative Percentage Deviation** (RPD) from the best-known solution:

$$\text{RPD} = \frac{\text{BestKnown} - \text{FinalCost}}{\text{BestKnown}} \times 100$$

An RPD of 0 means we matched the best-known value. A larger RPD means our solution is that many percent worse, so **lower is better** [5].

5.1. Exercise 1.1 — Iterative Improvement Results

Figure 1 shows solution quality at a glance for Insert, Exchange, and VND. Table 1 has the full numbers for all 12 configurations.

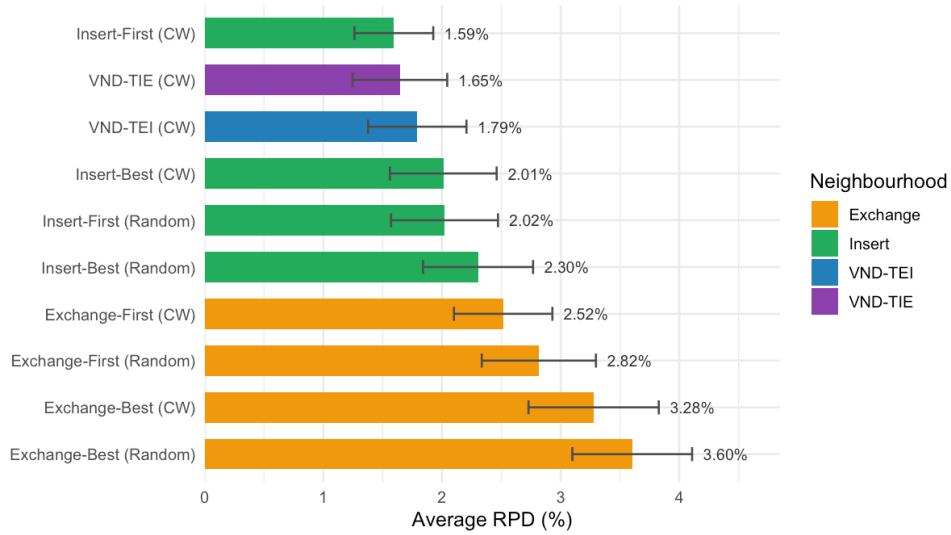


Figure 1: Average RPD (%) for Insert, Exchange and VND configurations, sorted from best to worst. Each bar shows the average over 78 instances. Transpose is excluded (RPD 19–35%) to keep the scale readable; it appears in Table 1.

Algorithm	Init	Avg RPD	SD	Avg Time (s)
Insert — First	CW	1.59	0.33	2.99
Insert — Best	CW	2.01	0.45	0.79
Insert — First	Random	2.02	0.45	4.28
Insert — Best	Random	2.30	0.46	0.85
Exchange — First	CW	2.52	0.41	3.97
Exchange — First	Random	2.82	0.48	4.87
Exchange — Best	CW	3.28	0.55	0.70
Exchange — Best	Random	3.60	0.50	0.76
Transpose — Best	CW	19.25	1.89	0.006
Transpose — First	CW	19.38	1.92	0.003
Transpose — Best	Random	34.47	3.80	0.007
Transpose — First	Random	34.61	3.80	0.004

Table 1: All 12 configurations over 78 instances, sorted by solution quality.

5.1.1. Observations

The **neighbourhood** is clearly the dominant factor. Insert is the best choice by a clear margin, with RPDs between 1.6% and 2.3% depending on the pivoting rule and initialisation. Exchange comes in second at 2.5–3.6%, and transpose is far behind at 19–35%. Transpose can only swap adjacent elements, so it needs an enormous number of moves to rearrange a permutation significantly — and it gets stuck long before reaching a decent solution.

Initialisation matters too, and the effect shows up in every configuration. CW consistently beats a random start, and the gap is largest for transpose (19% vs 34%). This makes sense: when a neighbourhood can only shift elements one position at a time, the starting point determines almost everything.

The more surprising result is the pivoting rule. For insert, **first-improvement finds better solutions than best-improvement** (1.59% vs 2.01% with CW). The reason has to do with the shape of the search space. Best-improvement always jumps to the steepest available gain — but the steepest gain is not always toward the global optimum.

It can commit to a local optimum early, with no way back. First-improvement is more conservative: it takes the first move that works, which keeps it from over-committing and gives it more room to explore before settling.

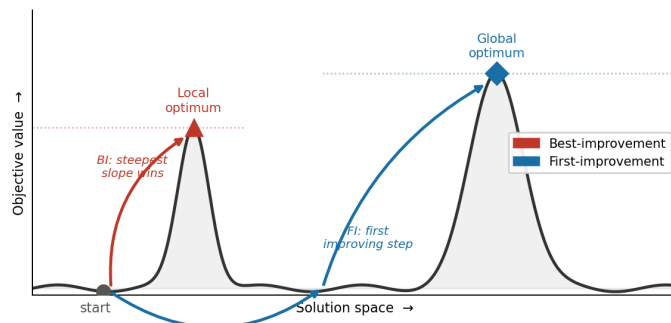


Figure 2: Difference between BI and FI

On time, though, the situation is reversed. Best-improvement (0.79 s for Insert-CW) is roughly 4× faster than first-improvement (2.99 s). This sounds odd given that BI inspects the full neighbourhood at every step, but the key is the number of iterations. Because BI always takes

the biggest gain available, it climbs faster and reaches a local optimum in far fewer steps. FI makes many small moves and the cumulative cost adds up. The time column in Table 1 shows this pattern clearly: FI variants are the slowest (red cells), BI variants sit in the middle (amber), and transpose barely registers (green).

5.1.2. Statistical Tests

Pairwise Wilcoxon signed-rank tests were applied at significance level $\alpha = 0.05$. A representative selection of results is shown in Table 2.

Algorithm A	Algorithm B	p-value	Significant?
Insert-First (CW)	Transpose-First (CW)	< 0.001	Yes
Insert-First (CW)	Exchange-First (CW)	< 0.001	Yes
Insert-First (CW)	Insert-Best (CW)	< 0.001	Yes
Insert-First (CW)	Insert-First (Random)	< 0.001	Yes
Insert-Best (CW)	Insert-First (Random)	0.9404	No
Exchange-First (CW)	Exchange-Best (CW)	< 0.001	Yes
Transpose-First (CW)	Transpose-First (Random)	< 0.001	Yes

Table 2: Selected pairwise Wilcoxon signed-rank test results ($\alpha = 0.05$). All 66 pairs were tested, nearly all are significant except the highlighted case.

Almost every pair is significantly different ($p < 0.001$), which is what we would expect given the large gaps in Table 1. The exception is **Insert-Best (CW) vs Insert-First (Random)** ($p = 0.94$): these two configurations use completely different strategies — one relies on a smart start and a weaker search, the other on a random start and a thorough one — yet they land at essentially the same quality (2.01% vs 2.02%). A good starting point can compensate for a weaker algorithm, at least in this case.

Figure 3 shows this visually: CW gives tighter, lower RPD distributions than a random start, regardless of neighbourhood or pivoting rule.

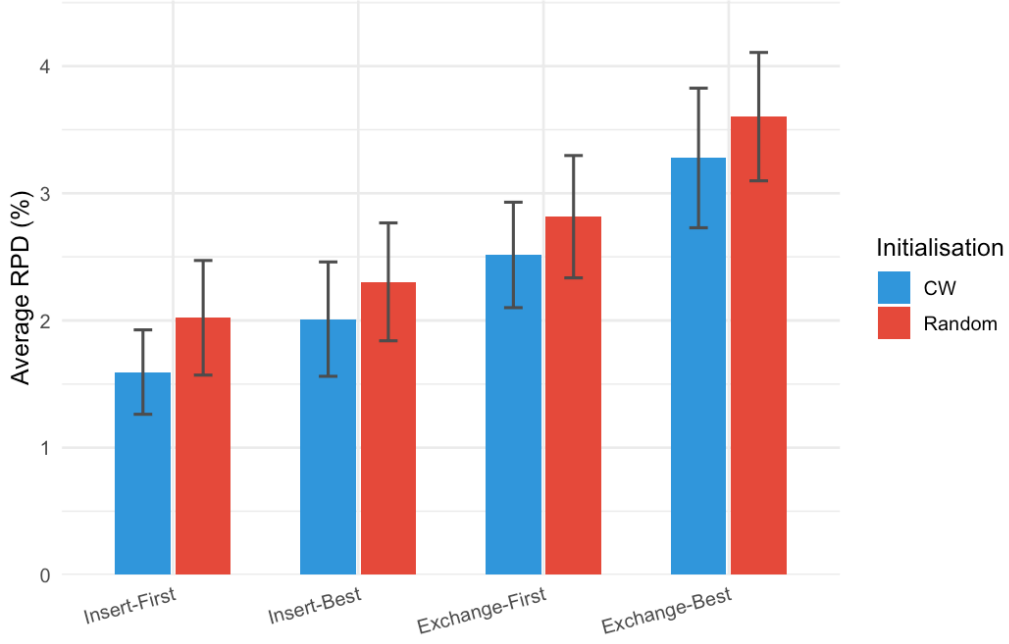


Figure 3: Average RPD (%) for CW initialisation and random initialisation, for each neighbourhood and pivoting rule combination.

5.2. Exercise 1.2 — VND Results

Table 3 and Figure 4 show the results for both VND orderings, starting from CW.

Algorithm	Init	Avg RPD	SD	Avg Time (s)	Total Time (s)
VND-TIE	CW	1.65	0.40	3.24	252.4
VND-TEI	CW	1.79	0.42	4.86	379.1

Table 3: VND results starting from the CW solution.

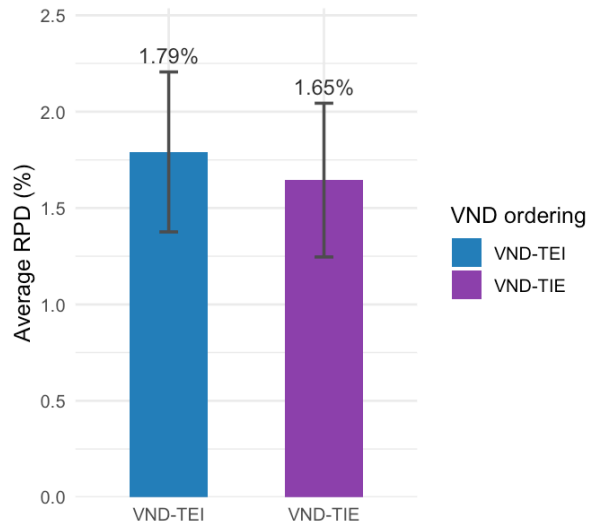


Figure 4: Average RPD (%) for VND-TEI (blue) and VND-TIE (purple), with ± 1 SD error bars. VND-TIE is lower and more consistent. The difference is statistically significant (Wilcoxon, $p = 0.0074$).

Both VND variants beat every single-neighbourhood algorithm, though the margin is thin. Insert-First (CW) achieves 1.59% RPD; VND-TIE gets 1.64%. Insert already captures most of what local search can offer here, so VND has little room to improve on it.

VND-TIE beats VND-TEI on both quality (1.64% vs 1.79%) and speed (3.24 s vs 4.86 s), and the difference is significant ($p = 0.0074$). The ordering matters: in TIE, insert runs before exchange, so the strongest neighbourhood gets to work on a solution that exchange has not already distorted. In TEI, exchange goes first and can trap the solution in a local optimum that insert struggles to escape. Which neighbourhood you run first is not a detail.

5.3. Execution Time and Complexity

5.3.1. Why do some algorithms take much longer than others?

The runtimes in Table 1 span several orders of magnitude — fractions of a millisecond for transpose, nearly 5 seconds for first-improvement insert. Per-iteration complexity explains the gap.

Each iteration scans the neighbourhood for an improving move. The cost depends on how many moves there are and how expensive each evaluation is:

Neighbourhood	Size	Cost per Δ	Cost per iteration
Transpose	$n - 1$	$O(1)$	$O(n)$
Exchange	$O\left(\frac{n^2}{2}\right)$	$O(n)$	$O(n^3)$ for BI, $O(n^2)$ avg for FI
Insert	$O(n^2)$	$O(n)$	$O(n^3)$ for BI, $O(n^2)$ avg for FI

Table 4: Theoretical cost per iteration of local search for each neighbourhood.

Transpose is $O(n)$ per iteration — only $n - 1$ pairs, each evaluated in $O(1)$ — which is why it finishes in under 10 ms even for $n = 250$. The cost is a tiny neighbourhood, and the results show it.

Exchange and Insert are $O(n^3)$ per iteration under best-improvement: $O(n^2)$ pairs, each costing $O(n)$. With first-improvement the average is lower since the scan stops on the first hit, but near a local optimum the algorithm checks almost everything before giving up, so the worst case is about the same.

Figure 5 shows the quality-time trade-off: the ideal is the bottom-left corner. Insert-FI (CW) wins on quality; Insert-BI and Exchange-BI trade some quality for speed.

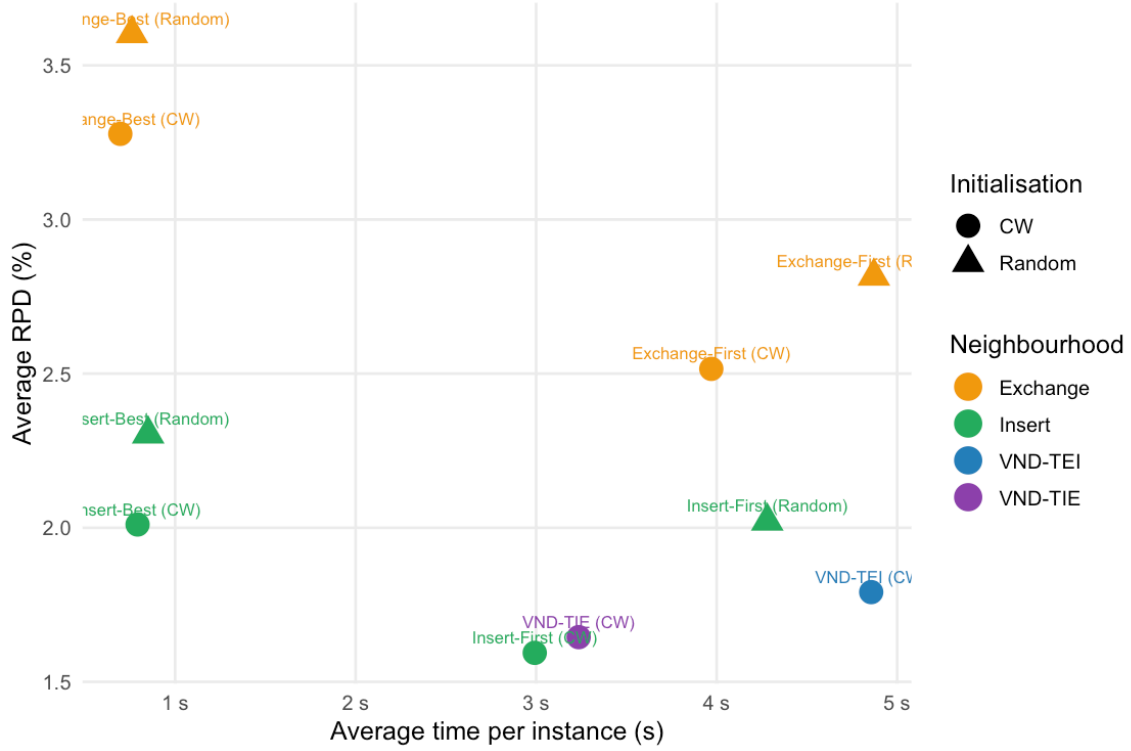


Figure 5: Quality–time trade-off for all configurations (Insert, Exchange and VND).

5.3.2. Why is best-improvement faster than first-improvement for insert?

BI takes 0.79 s, FI takes 2.99 s, yet BI checks the entire neighbourhood at every step. The reason is the number of iterations. BI always takes the largest available gain, so it converges in far fewer steps — the higher per-step cost is more than offset by doing fewer of them. FI moves in small increments and needs many more iterations to reach a local optimum.

5.3.3. Effect of instance size

From $n = 150$ to $n = 250$, the per-iteration cost grows roughly as $\left(\frac{250}{150}\right)^3 \approx 4.6$ for exchange and insert under best-improvement. The observed runtimes are consistent with this cubic scaling once we account for the mixed instance sizes in the benchmark.

5.3.4. Effect of initialisation on time

CW also speeds up convergence. Starting closer to a local optimum means fewer iterations to get there. For Insert-FI, the CW start cuts about 1.3 s off the runtime compared to a random start (2.99 s vs 4.28 s, a roughly $1.4\times$ speedup).

5.3.5. Possible optimisations

Three standard techniques could reduce runtime further:

- **Don’t-look bits:** skip elements not recently involved in an improving move.
- **Incremental delta table:** after a move, update only the affected gain values rather than recomputing the full table.
- **Candidate lists:** precompute a short list of the most promising move pairs and restrict the search to those.

6. Conclusion

All three design choices — neighbourhood, initialisation, pivoting rule — affect solution quality, but not by the same amount. The neighbourhood is by far the most important: insert dominates exchange, and both leave transpose far behind. Transpose just cannot cover enough ground with adjacent-only swaps, and it shows. CW initialisation helps everywhere, with the biggest payoff for transpose where the starting point matters most since the search itself is so weak. The pivoting rule has a smaller but consistent effect: first-improvement finds slightly better solutions at the cost of more time, while best-improvement trades a bit of quality for speed.

VND improves on the best single-neighbourhood algorithm, but barely — VND-TIE is 0.05% better than Insert-First (CW). Insert already captures most of what local search can offer on this problem. VND adds something, but not much. The neighbourhood ordering does matter though: TIE beats TEI in both quality and speed, and the difference is statistically significant. Running insert before exchange is clearly better than the other way around.

These algorithms will be the starting point for the second exercise, where metaheuristic techniques will push solution quality further by escaping local optima rather than settling into them.

References

- [1] Wikipedia contributors, “Standard deviation.” [Online]. Available: https://en.wikipedia.org/wiki/Standard_deviation
- [2] Wikipedia contributors, “Wilcoxon signed-rank test.” [Online]. Available: https://en.wikipedia.org/wiki/Wilcoxon_signed-rank_test
- [3] Wikipedia contributors, “P-value.” [Online]. Available: <https://en.wikipedia.org/wiki/P-value>
- [4] R Core Team, “wilcox.test: Wilcoxon Rank Sum and Signed Rank Tests.” [Online]. Available: <https://stat.ethz.ch/R-manual/R-devel/library/stats/html/wilcox.test.html>
- [5] T. Stützle, “Slides for INFO-H-413 Heuristic Optimization.” 2026.